文字接龙（三）-简单任务背后的复杂智能

- 来源: https://class.imooc.com/lesson/5254#mid=104875
- 阶段: 阶段一: 【认知基石】理解大模型的本质与工作原理
- 周: 第1周 语言模型是什么
- 模块: 文字接龙的游戏
- 课节: 图文 1-5 文字接龙（三）-简单任务背后的复杂智能

1-5 文字接龙（三）-简单任务背后的复杂智能

文字接龙（三）-简单任务背后的复杂智能

6.1 阅读指南

> 前两节，我们了解了ChatGPT如何玩文字接龙游戏，以及概率分布是如何计算的。 现在大模型的能力我们都有目共睹。但这里有一个深刻的悖论： 为什么一个看似简单的"预测下一个词"任务，能产生如此智能的表现？ 在这一节，我们将探讨这个悖论，并理解自回归机制及其缺陷。

6.2 简单的任务，复杂的智能

读到这里，可能有个疑问——第一次了解这个机制时，也有同样的疑问：

就这？预测下一个词？这么简单的任务，怎么能训练出ChatGPT这样看起来"智能"的系统？

这个疑问很合理。毕竟，"预测下一个词"听起来像是一个小学生都能理解的游戏，怎么会需要1750亿个参数？怎么会需要花费几百万美元训练？但实际上要把这个"简单"的任务做好，需要的能力一点都不简单。

例子一：理解多义词

看这个句子：

> “他拿起一只笔，笔_____”

这个问题怎么填?

这个问题的难点在于：“笔”这个词有歧义。

- 它可以是书写工具（笔尖、笔帽）
- 也可以是抽象概念（笔迹、笔法）

如果模型理解成物理对象，可能会填"尖"、“帽”； 如果理解成写作行为，应该填"迹"、“触”。

因为前面提到“拿起”，这是一个物理动作。一个好的模型，应该给“尖”或“帽”更高的概率。

这说明：模型必须理解词的多义性，必须根据上下文消歧。

例子二：指代消解

再看这个：

> “小明买了一部手机。它的屏幕_____”

“它”指的是什么？小明？还是手机?

显然是手机。所以下一个词应该是"很大"、“很清晰”，而不是"很聪明"（如果指小明）。

模型必须具备指代消解能力，才能预测准下一个词。

> [!NOTE] 指代消解（Coreference Resolution）：确定代词（如"它"、“他”、“这个”）指向哪个实体的能力。这是自然语言理解的核心能力之一。

例子三：常识推理

> “他把玻璃杯掉在地上，_____碎了。”

什么碎了？玻璃杯，还是地板?

这需要常识：玻璃杯易碎，地板不易碎。所以预测的应该是“玻璃杯”碎了，而非“地板”碎了。

模型必须具备物理世界的常识，才能做出正确预测。

例子四：语用理解

> “你能把窗户关上吗？我有点_____。”

下一个词应该是什么？“冷”？“热”？“累”？

这需要理解：问"能不能关窗"通常暗示"感觉冷"或"觉得吵"，而不太可能是"累"。

模型必须理解语言的语用规则，理解话语背后的隐含意图。

所以，要准确地预测下一个词，模型需要具备多种能力：理解语法结构、掌握语义关系、追踪指代消解、理解多义词消歧、具备世界知识、进行逻辑推理、理解语用规则。

这些能力，没有一个是"简单"的。

这就是为什么一个看似简单的"预测下一个词"的任务，能够训练出具有广泛语言理解能力的模型。因为要把这个任务做好，就必须理解语言的方方面面。

6.3 自回归：一个词滚雪球般生成一段话

这个词听起来很学术，但概念其实很直观。还记得前面“文字接龙”和“贪吃蛇游戏”的比喻吗？它的正式名字就是自回归。简单说：每一步预测出来的词，都会加入到上下文中，参与下一步的预测。

用函数的语言来描述这个过程。ChatGPT本质上是一个函数，每次调用接收输入，返回一个词：

```
1. 第1次调用：
   ChatGPT("今天天气怎么样？")
   → 返回：“今天”

2. 第2次调用（问题+上次输出）：
   ChatGPT("今天天气怎么样？今天")
   → 返回：“上海”

3. 第3次调用（问题+所有已生成的答案）：
   ChatGPT("今天天气怎么样？今天上海")
   → 返回：“的”

4. 第4次调用：
   ChatGPT("今天天气怎么样？今天上海的")
   → 返回：“天气”

...
```

注意一个关键细节：每次输入的不仅是已生成的答案，还包括最初的问题。

为什么要带上问题？因为如果只输入答案，ChatGPT就失去了上下文。只输入答案时，ChatGPT不知道在问天气还是日期，完全不知道要说什么。正确做法是问题加答案，知道是在回答天气问题，继续在天气语境下生成。

函数的输出，又被喂回函数作为输入的一部分。这就是“自回归”这个名字的来源——每一步的输出，都变成了下一步的输入的一部分。

6.4 为什么不一次性生成整个回答

答案有两个：

- 技术上更合适

一次性生成完整句子，需要同时预测几十上百个词，每个词都有接近几万种可能（看词表的大小），组合空间是天文数字。一个词一个词地生成，每次只需要做一次预测，简单得多。

- 更符合人类写作习惯

人类写文章也是这样的——写当前这句话时，基于前面已经写的内容；写下一句时，又基于前面所有的内容。

这种"滚雪球"式的生成方式，让模型能够生成任意长度的文本，只要上下文窗口允许。

6.5 自回归的缺陷：误差累积

自回归机制有一个微妙但重要的特性：误差会累积。比如：

误差会累积

```
第1步：输入“今天天气”
       正确预测：“真好”(60%)
       实际选择：“不错”(25%)
       ↓ 没问题，都是合理的
       |
       ▼
第2步：输入“今天天气不错”
       正确预测：“，可以”(50%)
       实际选择：“，我们”(15%)
       ↓ 开始有点偏离了
       |
       ▼
第3步：输入“今天天气不错，我们”
       正确预测：“出去走走”(40%)
       实际选择：“的心情”(10%)
       ↓ 越来越偏
       |
       ▼
第4步：输入“今天天气不错，我们的心情”
       预测：“也很好”
       (合理，但和最初的方向不同了)
```

每一步的小偏差，都会影响下一步的上下文，进而影响下下一步的预测。

就像走路偏离方向：

> 第1步：偏离1度，影响不大 第10步：累积偏离10度，有点歪了 第100步：累积偏离100度，完全走错方向了

这就是为什么有时ChatGPT会"写着写着就偏离了主题"。

如何缓解这个问题?

研究者们发现了几个方法：

方法一：更大的上下文窗口

上下文窗口是ChatGPT的“短期记忆”。它不能记住所有历史对话，只能记住最近的一段。这意味着文字接龙不能无限延伸下去。

```
生成文字的最大长度 = 上下文窗口大小 - 输入长度

1
```

例如，GPT-4的窗口是128,000词，输入2,000词，最多能生成126,000词，大约相当于9万多个中文字。

不同模型的窗口大小对比（2026年4月）：

模型	上下文窗口	约等于
GPT-5.5	1M 词	几本长篇小说
DeepSeek V4	1M 词	三本《三体》
Qwen3.6-Plus	1M 词	几本长篇小说
Kimi K2.6	256K 词	一部长篇小说
Gemini 3.1 Pro	1M 词	几本长篇小说
GLM-5.1	200K 词	半本长篇小说

> [!NOTE] 截至2026年4月，主流模型的上下文窗口已达到100万词（DeepSeek V4、Qwen3.6-Plus、Gemini 3.1 Pro）。但研究发现，当上下文超过12.8万词时，大部分模型的性能会显著下降。因此，虽然窗口在变大，但有效窗口（模型能稳定工作的范围）仍是一个需要关注的指标。

窗口越大，能"看到"更长的历史对话，更好地理解上下文逻辑，减少因为"忘记"而导致的偏离。

方法二：更好的采样策略

不要完全随机选择，而是在“质量”和“多样性”之间平衡，例如Top-p采样、Temperature调节（后面会详细讲）。

方法三：加入外部记忆

不只依赖生成的文本本身，还引入外部知识库，例如RAG（检索增强生成，后面有一章专门讲这个）。

方法四：思维链（CoT， Chain of Thought）

让模型先“思考”，再回答，减少直接一路生成导致的偏离。这就是思维链CoT（Chain of Thought），后面章节会详细讲。

但总体而言，自回归虽然有很多问题，但它仍然是目前大语言模型的主流方法，因为它的优势大于劣势。而那些劣势，正在通过各种技术手段逐步消解。

6.6 ■ 学点英语

中文	English	音标	说明
指代消解	Coreference Resolution	/ˈkoʊrefərəns ˌrezəˈluːʃən/	确定代词指向哪个实体的语言理解能力
上下文窗口	Context Window	/ˈkɒntekst ˈwɪndəʊ/	模型能同时处理的文本长度上限
思维链	Chain of Thought (CoT)	/tʃeɪn ɒv θɔːt/	让模型先逐步推理再给出最终回答的方法
检索增强生成	Retrieval-Augmented Generation (RAG)	/rɪˈtriːvəl ˈɔːgmentɪd ˌdʒenəˈreɪʃən/	引入外部知识库辅助生成的技术